

WEEKLY ASSESSMENT - 2

Python Programming Solutions Manual

1. Demonstrate how to join a list of strings into a single sentence.

To combine a list of strings into a single sentence, Python provides the built-in `.join()` method belonging to string objects. The syntax is `separator.join(list_of_strings)`, where the separator string is placed between each element.

```
words = ["Python", "is", "a", "powerful", "programming", "language."]
# Join using a single space as the separator
sentence = " ".join(words)
print(sentence)
```

Output:

```
Python is a powerful programming language.
```

2. Write a code snippet to remove duplicates from a list using a set.

A `set` in Python is an unordered collection of unique elements. Passing a duplicate-filled list to the `set()` constructor automatically eliminates all redundant entries. To return it back to its original data structure, you can wrap it back inside a `list()` constructor.

```
original_list = [1, 2, 2, 3, 4, 4, 4, 5, 1, 6]
# Convert to set to remove duplicates, then back to a list
unique_list = list(set(original_list))
print(unique_list)
```

Output:

```
[1, 2, 3, 4, 5, 6]
```

3. How do you check if a specific key exists in a dictionary?

The most idiomatic and efficient way to check if a key exists in a dictionary is by using the `in` membership keyword. Alternatively, you can use the `.get()` method, which returns `None` (or a custom default value) if the key is not found.

```
student_info = {"name": "Ajay", "course": "BBA", "age": 19}

# Method 1: Using the 'in' keyword
if "course" in student_info:
    print("Key 'course' exists in the dictionary.")

# Method 2: Using the .get() method
if student_info.get("grade") is None:
    print("Key 'grade' does not exist.")
```

Output:

```
Key 'course' exists in the dictionary.
Key 'grade' does not exist.
```

4. Explain the difference between `is` and `==`.

Although they may look similar, `is` and `==` serve fundamentally different purposes in Python object comparison:

- **`==` (Equality Operator):** Checks whether the **values** of the two objects are equal. It returns `True` if the contents are identical.
- **`is` (Identity Operator):** Checks whether both variables point to the exact same **memory location** (**identity**). It returns `True` only if `id(a) == id(b)`.

```
list_a = [1, 2, 3]
list_b = [1, 2, 3]
list_c = list_a

print(list_a == list_b) # True: They contain the exact same items
print(list_a is list_b) # False: They are distinct objects in memory
print(list_a is list_c) # True: Both variables reference the same object
```

Output:

```
True
False
True
```

5. Create a simple calculator using if-elif-else logic.

Below is a multi-purpose script that evaluates an operator symbol against conditional statements to perform basic mathematical operations (+, -, *, /).

```
num1 = 12
num2 = 4
operator = "/"

if operator == "+":
    result = num1 + num2
elif operator == "-":
    result = num1 - num2
elif operator == "*":
    result = num1 * num2
elif operator == "/":
    if num2 != 0:
        result = num1 / num2
    else:
        result = "Error! Division by zero."
else:
    result = "Invalid Operator"

print(f"{num1} {operator} {num2} = {result}")
```

Output:

```
12 / 4 = 3.0
```

6. Write a program to find the largest number in a list without using max().

To find the highest number manually, initialize a variable to hold the first element of the list, then iterate through the remaining items updating the variable whenever a larger value is encountered.

```
numbers = [23, 78, 45, 92, 56, 12]

# Initialize with the first element
largest = numbers[0]

for num in numbers:
    if num > largest:
        largest = num

print("The largest number is:", largest)
```

Output:

```
The largest number is: 92
```

7. Demonstrate how to use the .strip() method on a messy input string.

The `.strip()` method removes leading and trailing whitespaces (newlines, tabs, spaces) from a string. You can also supply optional character sequences as arguments to trim specific unwanted characters from either end.

```
messy_input = "
    Python Programming Frameworks!!!
"
clean_input = messy_input.strip()

print(f"Original: '{messy_input}'")
print(f"Cleaned: '{clean_input}'")
```

Output:

```
Original: '
    Python Programming Frameworks!!!
'
Cleaned: 'Python Programming Frameworks!!!'
```

8. How do you convert a float to an integer? What happens to the decimal?

You can cast a float value to an integer using Python's built-in `int()` constructor function. Doing this does **not round** the value to the nearest whole number; instead, it completely discards (truncates) the decimal part.

```
pi_val = 3.14159
negative_val = -5.99

print(int(pi_val))          # Truncates decimal, resulting in 3
print(int(negative_val))    # Truncates decimal, moving towards 0 resulting in -5
```

Output:

```
3
-5
```

9. Write a script that prints the multiplication table of a given number.

A simple `for` loop utilizing the `range()` function can loop from 1 through 10 to generate a sequential arithmetic progression table.

```
number = 7

print(f"Multiplication Table for {number}:")
for i in range(1, 11):
    print(f"{number} x {i} = {number * i}")
```

Output:

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

10. Explain the use of the `in` keyword in Python.

The `in` keyword in Python has two primary application contexts:

1. **Membership Testing:** Checks whether a value exists inside an iterable series (such as strings, lists, tuples, sets, or dictionaries). It returns `True` or `False`.
2. **Loop Iteration:** Used in `for` loops to step sequentially through each item of an iterable sequence.

```
# Membership checking
message = "Welcome to Python"
print("Python" in message) # True

# Sequence Iteration
fruits = ["Apple", "Banana", "Cherry"]
for fruit in fruits:
    print(fruit)
```

Output:

```
True
Apple
Banana
Cherry
```